

1545. Find Kth Bit in Nth Binary String

Solved 

Medium

 Topics

 Companies

 Hint

Given two positive integers n and k , the binary string S_n is formed as follows:

- $S_1 = "0"$
- $S_i = S_{i-1} + "1" + \text{reverse}(\text{invert}(S_{i-1}))$ for $i > 1$

Where $+$ denotes the concatenation operation, $\text{reverse}(x)$ returns the reversed string x , and $\text{invert}(x)$ inverts all the bits in x (0 changes to 1 and 1 changes to 0).

For example, the first four strings in the above sequence are:

- $S_1 = "0"$
- $S_2 = "011"$
- $S_3 = "0111001"$
- $S_4 = "011100110110001"$

Return *the k^{th} bit in S_n* . It is guaranteed that k is valid for the given n .

Example 1:

Input: $n = 3, k = 1$

Output: "0"

Explanation: S_3 is "0111001".

The 1st bit is "0".

Example 2:

Input: $n = 4, k = 11$

Output: "1"

Explanation: S_4 is "011100110110001".

The 11th bit is "1".

Constraints:

- $1 \leq n \leq 20$
- $1 \leq k \leq 2^n - 1$

Python:

class Solution:

def findKthBit(self, n: int, k: int) -> str:

if n == 1:
 return '0'

length = (1 << n) - 1
mid = (length + 1) // 2

if k == mid:
 return '1'

if k < mid:
 return self.findKthBit(n - 1, k)

c = self.findKthBit(n - 1, length - k + 1)
return '1' if c == '0' else '0'

JavaScript:

```
var findKthBit = function(n, k) {  
    if (n === 1) return '0';  
  
    const length = (1 << n) - 1;  
    const mid = Math.floor((length + 1) / 2);  
  
    if (k === mid) return '1';  
    if (k < mid) return findKthBit(n - 1, k);  
  
    const c = findKthBit(n - 1, length - k + 1);  
    return c === '0' ? '1' : '0';  
};
```

Java:

```
class Solution {  
    public char findKthBit(int n, int k) {  
        if (n == 1) return '0';  
  
        int len = (1 << n) - 1;  
        int mid = (len + 1) / 2;  
  
        if (k == mid) return '1';  
        if (k < mid) return findKthBit(n - 1, k);  
  
        char c = findKthBit(n - 1, len - k + 1);  
        return c == '0' ? '1' : '0';  
    }  
}
```